

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I A G Jayapradhan (192471016) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A G Jayapradhan

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments

- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

1. AI-Powered Drone for Emergency Medicine Delivery (Greedy Best-First Search & A*)

i. Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the next path based only on the heuristic value $h(n)$ (estimated distance to the destination). It always chooses the node that appears closest to the goal.

Behavior in this scenario

- Uses straight-line distance to select routes.
- Ignores actual travel cost caused by weather or difficult terrain.
- Quickly finds a possible path for emergency delivery.
- May choose blocked or risky routes because it does not consider movement cost.

Strengths

- Very fast decision making.
- Requires less memory.
- Suitable when immediate response is more important than optimality.

Weaknesses

- Does not guarantee the shortest or safest route.
 - Can get trapped if heuristic is misleading.
 - Performs poorly in uncertain and dynamic environments.
-

ii. A* Search

A* evaluates each node using

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ = Actual cost from start to current node
- $h(n)$ = Estimated cost from current node to goal

Behavior

- Considers both travel cost and estimated remaining distance.
 - Avoids routes with high movement cost due to floods or bad weather.
 - Recalculates better paths when updates about blocked roads are received.
 - Produces safer and more efficient routes.
-

iii. Impact of Heuristic Accuracy

For Greedy Best-First Search

- Accurate heuristic → Very fast and good solution.
- Poor heuristic → Wrong path selection and longer travel time.

For A*

- Accurate heuristic → Fast and optimal.
 - Less accurate heuristic → More nodes explored but still finds optimal path (if heuristic is admissible).
-

iv. Effect of Dynamic Changes (Blocked Paths)

Greedy Best-First Search

- May continue toward blocked roads.
- Needs repeated replanning.
- Performance decreases significantly.

A*

- Updates path costs when obstacles appear.
- Can recompute an alternative route.
- More reliable in changing environments.

v. Recommended Algorithm

Recommended: A*

Justification

- Produces optimal routes.
- Balances actual travel cost and estimated distance.
- Handles dynamic updates more effectively.
- Improves safety by avoiding risky routes.
- Better suited for emergency medicine delivery despite slightly higher computation.

2. Smart City Traffic Optimization (Hill Climbing & Simulated Annealing / Genetic Algorithm)

Problem Formulation

Objective

Minimize:

- Total waiting time
- Fuel consumption
- Traffic congestion

Variables

- Green signal duration
- Red signal duration
- Signal sequence

Why Traditional Search is Inefficient

- Thousands of intersections create millions of possible combinations.
- Traffic changes continuously.
- Exhaustive search requires enormous computation.
- Real-time optimization becomes impossible.

i. Hill Climbing

Hill Climbing starts with one signal timing configuration and repeatedly moves to a neighboring solution that improves traffic flow.

Advantages

- Simple implementation.
- Fast computation.

- Suitable for local optimization.

Limitations

- Can stop at local optimum.
 - Cannot escape poor solutions.
 - Depends heavily on starting point.
-

ii. Local Maxima, Plateaus and Ridges

Local Maximum

A solution better than nearby solutions but not globally best.

Example: One busy junction improves while the citywide traffic remains poor.

Plateau

Many neighboring solutions have identical quality.

Example: Changing signal timing by a few seconds produces no noticeable improvement.

Ridge

The best improvement requires moving in multiple directions simultaneously.

Example: Several intersections must change together to reduce congestion.

iii. Simulated Annealing and Genetic Algorithm

Simulated Annealing

- Sometimes accepts worse solutions.
- Escapes local maxima.
- Probability decreases gradually.

Advantages

- Finds better global solutions.
 - Avoids getting stuck.
-

Genetic Algorithm

Works like biological evolution.

Steps:

1. Generate population.
2. Evaluate fitness.
3. Select best solutions.
4. Perform crossover.
5. Apply mutation.
6. Repeat.

Advantages

- Explores many solutions simultaneously.
 - Highly scalable.
 - Suitable for complex optimization.
-

iv. Recommended Approach

Recommended: Genetic Algorithm

Justification

- Handles very large search spaces.
 - Adapts to changing traffic conditions.
 - Finds near-optimal signal timings.
 - More scalable than Hill Climbing.
 - Can optimize multiple objectives simultaneously.
-

3. Mars Rover Exploration (Online Search Agent)

i. Why Online Search?

The rover has:

- No complete map.
- Limited sensing.
- Unknown terrain.
- Learns while moving.

Offline search requires complete knowledge beforehand, which is unavailable. Therefore, an **Online Search Agent** is required.

ii. Challenges

Partial Observability

- Rover sees only nearby terrain.
- Hazards remain unknown until detected.

Dynamic Environment

- Terrain conditions may change.
 - New obstacles appear.
 - Communication delay prevents constant human control.
-

iii. Internal Model

The rover continuously updates its map.

Steps:

1. Sense surroundings.
2. Detect obstacles.
3. Update internal map.
4. Plan next movement.
5. Repeat until destination.

This learning process improves navigation.

iv. Comparison

Feature	Uninformed Search	Informed Search	Online Search
Complete map required	Yes	Yes	No
Uses heuristic	No	Yes	May use
Handles unknown world	No	Limited	Yes
Learns during search	No	No	Yes
Suitable for Mars	No	Partly	Yes

v. Recommended Approach

Recommended: Online Search Agent

Justification

- Operates without complete knowledge.
 - Continuously adapts.
 - Safely avoids hazards.
 - Suitable for uncertain environments.
 - Supports autonomous exploration.
-

4. University Exam Timetable (Constraint Satisfaction Problem)

CSP Formulation

Variables

Each course examination.

Example:

- C1
 - C2
 - C3
-

Domains

Available:

- Dates
 - Time slots
 - Exam halls
-

Constraints

- No student has overlapping exams.
 - Hall capacity must not exceed limits.
 - Faculty must be available.
 - Some exams occur before others.
 - Special accommodation students receive appropriate halls.
-

i. Variables, Domains and Constraints

Variables

- Exams

Domains

- Available dates
- Time slots
- Rooms

Constraints

- Student conflicts
 - Hall capacity
 - Faculty schedule
 - Subject order
 - Accessibility requirements
-

ii. Backtracking Search

Steps:

1. Select an exam.
 2. Assign a time slot.
 3. Check constraints.
 4. If violation occurs, undo assignment.
 5. Try another slot.
 6. Continue until all exams are scheduled.
-

iii. Constraint Propagation (Forward Checking)

Forward checking:

- Removes invalid future choices immediately.
- Detects conflicts early.
- Reduces unnecessary search.
- Improves efficiency.

Example:

If Hall A is assigned at 9 AM, remove Hall A from other exams at 9 AM.

iv. Recommended Strategy

Recommended: Backtracking + Forward Checking

Justification

- Produces valid schedules.
 - Reduces search space.
 - Efficient for large universities.
 - Handles multiple constraints effectively.
 - Scales better than simple backtracking.
-

5. Strategic Game AI (Minimax & Alpha-Beta Pruning)

i. Minimax Algorithm

Minimax assumes:

- AI (MAX) tries to maximize score.
- Human player (MIN) tries to minimize AI's score.

The algorithm explores the game tree and selects the move with the best guaranteed outcome.

ii. Computational Challenges

Large game trees cause:

- Huge branching factor.
 - High memory usage.
 - Long computation time.
 - Unsuitable for real-time decisions without optimization.
-

iii. Alpha-Beta Pruning

Alpha-Beta Pruning skips branches that cannot improve the final decision.

Alpha (α)

Best value MAX can guarantee.

Beta (β)

Best value MIN can guarantee.

If $\alpha \geq \beta$, remaining branches are ignored because they cannot affect the result.

Advantages

- Reduces nodes explored.
 - Faster search.
 - Produces the same optimal move as Minimax.
 - Enables deeper search within the same time.
-

iv. Evaluation Function

Example:

Evaluation Score =

- $+50 \times \text{Resources}$
- $+40 \times \text{Army Strength}$
- $+30 \times \text{Territory Control}$
- $+20 \times \text{Unit Health}$
- $+10 \times \text{Strategic Position}$
- $-40 \times \text{Enemy Threat}$

Factors Considered

- Resources available
- Number of units
- Unit health
- Map control
- Defensive position
- Enemy strength

These factors estimate how favorable the current game state is for the AI.

v. Recommended Strategy

Recommended: Alpha-Beta Pruning with Minimax

Justification

- Maintains optimal decision quality.
- Significantly reduces computation.
- Enables faster responses under strict time limits.
- Supports deeper search in large game trees.
- Well suited for real-time strategy games.

Conclusion (Common Exam Conclusion)

Different AI techniques are suitable for different types of problems:

- **A*** is best for dynamic pathfinding because it balances actual cost and heuristic estimates.
- **Genetic Algorithms** are ideal for large-scale optimization problems such as traffic management.
- **Online Search Agents** are essential for exploration in unknown environments like Mars.
- **Backtracking with Forward Checking** efficiently solves complex scheduling problems modeled as CSPs.
- **Minimax with Alpha-Beta Pruning** provides strong real-time game decision-making by reducing unnecessary search while maintaining optimal play.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand